

แบบฝึกปฏิบัติครั้งที่ 3

Data Definition Language (DDL) และการจัดการข้อมูลเบื้องต้น (DML)

1. วัตถุประสงค์ของบทเรียน

ผู้เรียนจะสามารถ:

1. ฝึกปฏิบัติการสร้างฐานข้อมูล (CREATE DATABASE)
2. ฝึกปฏิบัติการลบฐานข้อมูล (DROP DATABASE)
3. ฝึกปฏิบัติการสร้างตาราง แก้ไขตาราง เพิ่ม/ลบ/แก้ไข/เปลี่ยนชื่อคอลัมน์ ด้วยคำสั่ง CREATE TABLE / ALTER TABLE
4. ฝึกปฏิบัติการเพิ่ม ลบ แก้ไขข้อมูล ด้วยคำสั่ง INSERT / DELETE / UPDATE

2. คำสั่งในการจัดการฐานข้อมูล

2.1 CREATE DATABASE — สร้างฐานข้อมูล

```
CREATE DATABASE database_name;
```

ตัวอย่าง:

```
CREATE DATABASE labdb;
```

เลือกใช้ฐานข้อมูล:

2.2 DROP DATABASE — ลบฐานข้อมูล

```
DROP DATABASE database_name;
```

ตัวอย่าง:

```
DROP DATABASE labdb;
```

3. คำสั่งสร้างตาราง (CREATE TABLE)

โครงสร้างคำสั่ง:

```
CREATE TABLE table_name (
    column_name_1 data_type [column_constraint],
    column_name_2 data_type [column_constraint],
    ...
    PRIMARY KEY (column_name),
    FOREIGN KEY (column_name) REFERENCES other_table(column_name)
);
```

คำอธิบาย:

- **table_name** = ชื่อตาราง
- **data_type** = ประเภทข้อมูล เช่น INT, CHAR(n), VARCHAR(n), DATE
- Constraint คือ “กฎ (rule)” ที่ใช้ ควบคุมความถูกต้องของข้อมูล ในตาราง เพื่อให้ข้อมูลที่ถูกบันทึก: ถูกต้องตามเงื่อนไข ไม่ขัดแย้งกัน รักษาความสัมพันธ์ของข้อมูล

ประเภทของ Constraint ที่ใช้บ่อยใน PostgreSQL

Constraint	หน้าที่
PRIMARY KEY	ระบุแถวไม่ให้ซ้ำ
FOREIGN KEY	เชื่อมความสัมพันธ์ระหว่างตาราง FOREIGN KEY ... ON DELETE CASCADE เป็นต้น
UNIQUE	ห้ามข้อมูลซ้ำ
NOT NULL	ห้ามเป็นค่าว่าง NULL, NOT NULL
CHECK	กำหนดเงื่อนไขของค่า
DEFAULT	กำหนดค่าเริ่มต้น

4. การใส่ Constraint

4.1 ใส่ Constraint ระดับคอลัมน์ (Column-level)

```
CREATE TABLE client (
    client_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    prefNoRooms INT CHECK (prefNoRooms > 0)
);
```

อธิบาย:

PRIMARY KEY → ห้ามซ้ำ + ห้าม NULL

NOT NULL → ต้องมีค่า

UNIQUE → ห้ามซ้ำ

CHECK → ตรวจสอบเงื่อนไข

4.2 ใส่ Constraint ระดับตาราง (Table-level)

```
CREATE TABLE booking (
    Booking_id SERIAL,
    client_id INT,
    room_no INT,
    booking_date DATE,
    CONSTRAINT pk_booking PRIMARY KEY (booking_id),
    CONSTRAINT fk_client
        FOREIGN KEY (client_id)
        REFERENCES client(client_id),
    CONSTRAINT chk_room CHECK (room_no > 0)
);
```

จุดเด่น:

ตั้งชื่อ constraint ชัดเจน (pk_booking, fk_client)

เหมาะสำหรับ Composite Key และ Foreign Key

4.3 การเพิ่ม Constraint หลังจากสร้างตารางแล้ว (ALTER TABLE)

โครงสร้างทั่วไป:

```
ALTER TABLE table_name
ADD CONSTRAINT constraint_name constraint_definition;
```

4.4 ตัวอย่างเพิ่ม PRIMARY KEY

```
ALTER TABLE client
ADD CONSTRAINT pk_client
PRIMARY KEY (client_id);
```

4.2 ตัวอย่างเพิ่ม UNIQUE

```
ALTER TABLE client
ADD CONSTRAINT uq_client_email
UNIQUE (email);
```

4.3 ตัวอย่างเพิ่ม CHECK

```
ALTER TABLE client
ADD CONSTRAINT chk_pref_rooms
CHECK (prefNoRooms > 0);
```

4.4 ตัวอย่างเพิ่ม FOREIGN KEY

```
ALTER TABLE booking
ADD CONSTRAINT fk_booking_client
FOREIGN KEY (client_id)
REFERENCES client(client_id);
```

4.5 ถ้าต้องการ “เพิ่มคอลัมน์”

```
ALTER TABLE client
ADD prefNoRooms INT;
```

4.6 ถ้าต้องการ “เพิ่ม constraint ให้คอลัมน์นั้น”

```
ALTER TABLE client
ADD CONSTRAINT chk_pref_rooms
CHECK (prefNoRooms > 0);
```

4.7 หรือเขียนรวมตอนเพิ่มคอลัมน์เลย:

```
ALTER TABLE client
ADD prefNoRooms INT CHECK (prefNoRooms > 0);
```

เปรียบเทียบ: CREATE TABLE vs ALTER TABLE

ประเด็น	CREATE TABLE	ALTER TABLE
เวลาใช้	ตอนสร้างตาราง	หลังสร้างแล้ว
ความเหมาะสม	ออกแบบใหม่	ปรับปรุงภายหลัง
Performance	ดีกว่า	อาจกระทบข้อมูลเดิม
ความเสี่ยง	ต่ำ	อาจ error ถ้าข้อมูลไม่ผ่าน constraint

“ควรออกแบบ constraint ให้ครบตั้งแต่ CREATE TABLE”

5.4 ลบ constraint

```
ALTER TABLE table_name
DROP CONSTRAINT constraint_name;
```

5.5 กำหนดค่า Default ของคอลัมน์

```
ALTER TABLE table_name
```

```
ALTER column_name SET DEFAULT value;
```

ตัวอย่าง:

```
ALTER TABLE Staff ALTER sex SET DEFAULT 'F';
```

5.6 ลบค่า Default

```
ALTER TABLE table_name
```

```
ALTER column_name DROP DEFAULT;
```

5.7 เปลี่ยนชื่อตาราง

```
ALTER TABLE old_name RENAME new_name;
```

หรือ

```
ALTER TABLE old_name TO new_name;
```

ตัวอย่าง:

```
ALTER TABLE employee TO employees;
```

5.8 แก้ไขชนิดข้อมูลของคอลัมน์

```
ALTER TABLE table_name
```

```
MODIFY COLUMN column_name data_type;
```

ตัวอย่าง:

```
ALTER TABLE employee MODIFY COLUMN age INT(3);
```

5.9 เปลี่ยนชื่อคอลัมน์

```
ALTER TABLE table_name
```

```
RENAME COLUMN old_name TO new_name;
```

ตัวอย่าง:

```
ALTER TABLE employee RENAME COLUMN fname TO firstname;
```

5. คำสั่งลบตาราง

```
DROP TABLE table_name;
```

ตัวอย่าง:

```
DROP TABLE student;
```

6. คำสั่งจัดการข้อมูล (DML)

6.1 INSERT — เพิ่มข้อมูล

โครงสร้าง:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

หมายเหตุสำคัญ

- ตัวอักษรต้องใส่ใน ' ' เช่น 'Test'
- ตัวเลขไม่ต้องใส่ ' ' เช่น 100

ตัวอย่าง:

```
INSERT INTO persons (Person_id, RateHour, Person_name, address, BirthDate)
VALUES ('001',1000,'นัชชา สীগัน','ท่าศาลา เมืองเหนือ','1975-01-03');
```

แบบย่อ (ลำดับตรงกับโครงสร้าง):

```
INSERT INTO persons
VALUES ('001',1000,'นัชชา สীগัน','ท่าศาลา เมืองเหนือ','1975-01-03');
```

6.2 UPDATE — แก้ไขข้อมูล

โครงสร้าง:

```
UPDATE table_name
SET column1=value1, column2=value2
WHERE condition;
```

ตัวอย่าง:

```
UPDATE persons
SET RateHour=1000, Person_name='นัชชา แต่งงานแล้ว'
WHERE Person_id='001';
```

6.3 DELETE — ลบข้อมูล

โครงสร้าง:

```
DELETE FROM table_name
WHERE condition;
```

ตัวอย่าง:

```
DELETE FROM persons
WHERE Person_id='001';
```

7. แบบฝึกปฏิบัติ

ก่อนเริ่ม ให้สร้างฐานข้อมูลและเลือกใช้งาน:

```
CREATE DATABASE labddl;
```

ข้อ 1 — สร้างตาราง Employee

ให้สร้างตาราง **Employee** ตามโครงสร้างที่กำหนด

ข้อ 2 — เปลี่ยนชื่อตารางเป็น Employees

```
ALTER TABLE Employee RENAME Employees;
```

ข้อ 3 — เพิ่ม/แก้ไขคอลัมน์ให้มีคุณสมบัติครบถ้วน

(ตามตารางคุณสมบัติที่อาจารย์กำหนด เช่น NOT NULL, PRIMARY KEY, DEFAULT ฯลฯ)

ข้อ 4 — เพิ่มข้อมูลตัวอย่าง

ใส่ข้อมูลตามรูปแบบต่อไปนี้

employee_id	firstname	lastname	department_id	address	ratehour
-------------	-----------	----------	---------------	---------	----------

นักศึกษาต้องเขียนคำสั่ง INSERT ให้ครบทุก row

ส่งงานโดยใช้คำสั่ง

```
SELECT * FROM employees;
```

ข้อ 5 — เปลี่ยนชนิดข้อมูลของ Department_id จาก INT เป็น CHAR(1)

ผู้เรียนต้องคิดเองว่าต้อง:

1. ลบ constraint ที่เกี่ยวข้องหรือไม่
2. ตรวจสอบข้อมูลเดิม
3. แปลงข้อมูลก่อนหรือหลัง
4. ใช้คำสั่ง MODIFY

คำตอบตัวอย่าง:

```
ALTER TABLE employees MODIFY COLUMN department_id CHAR(1);
```

ข้อ 6 — แก้ไขค่า Department_id ตามเงื่อนไข

- employee_id 1, 3, 4 → department_id = '4'
- employee_id 5, 6, 7, 8 → department_id = '2'

ตัวอย่างคำสั่ง:

```
UPDATE employees SET department_id='4' WHERE employee_id IN (1,3,4);
```

```
UPDATE employees SET department_id='2' WHERE employee_id IN (5,6,7,8);
```

ข้อ 7 — ลบคนชื่อ Aumnath

```
DELETE FROM employees
```

```
WHERE firstname='Aumnath';
```

ข้อ 8 — แสดงพนักงานที่อยู่ตำบล Tasala Mauangthai

```
SELECT * FROM employees
WHERE address LIKE '%Tasala Mauangthai%';
```

ตัวอย่างเพิ่มเติม

```
CREATE TABLE dept (
    name CHAR(10) NOT NULL,
    location CHAR(20),
    CONSTRAINT dept_unique UNIQUE(name)
);
```

```
CREATE TABLE emp (
    name CHAR(10) NOT NULL,
    salary DECIMAL(10,2),
    dept CHAR(10),
    CONSTRAINT empref REFERENCES dept(name)
);
```

ลบ constraint:

```
ALTER TABLE dept DROP CONSTRAINT dept_unique RESTRICT;
ALTER TABLE dept DROP CONSTRAINT dept_unique CASCADE;
```